

Lab 21 - Packaging and Delivery: Helm, Kustomize and GitOps

Field	Value
Lab id	Lab 21
Course	TGS-2023039343 Kubernetes and Cloud Native Associate (KCNA) Training v6.0
Day / Topic	Day 4 Cloud Native Application Delivery
Duration	55 minutes
Namespace	kcna-lab21
Maps to	LO5 Demonstrate Kubernetes solution for a specific business problem A5 Demonstrate how the recommended IT solutions and components collectively address an existing business problem or need K5 Tools and techniques for solution architecture modelling

Tooling note - read this first

helm is not installed on this machine and may not be on yours. That is deliberate, and this lab is built around it:

* Everything graded runs offline with kubectl alone. kubectl kustomize is built into kubectl (it embeds Kustomize v5) and renders base + overlays to stdout with no cluster and no network. verification/checks.sh grades the *static render*.

* The Helm sections are still fully authored and readable. You will read chart/ field by field, which is most of the learning. If you have helm, the render commands work exactly as printed and the checks grade them too. If you do not, checks.sh reports [SKIP] for those lines and still passes.

Install Helm if you want the full experience:

```
helm version
```

```
command not found: helm
```

Platform	Command	
macOS (Homebrew)	brew install helm	
Linux (script)	`curl -fsSL https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3 \`	bash`
Any	< https://helm.sh/docs/intro/install/ >	

Argo CD and Flux are not installed either, and this lab does not install them. The reconciliation model is taught by reading a real Application manifest field by field - which is what you would do on the job before approving one. Those files live in gitops/, outside manifests/, precisely so that kubectl apply -f manifests/ cannot pick up a kind whose CRD is absent.

1. Objective

By the end of this lab you will be able to:

1. Explain why hand-maintained YAML fails at more than one environment, in terms of the specific failure it produces.
2. Read a Helm chart and name the purpose of Chart.yaml, values.yaml, templates/, _helpers.tpl and NOTES.txt, including the difference between version and appVersion.
3. State Helm's values precedence and predict the rendered output of helm template given a chart and a values file.
4. Explain the checksum/config annotation and the failure it prevents.
5. Build a Kustomize base and two overlays, and render both with kubectl kustomize - with no cluster.
6. Choose between a strategic-merge patch and a JSON 6902 patch, and say what each can do that the other cannot.
7. Compare templating (Helm) with patching (Kustomize) and defend a choice for a given situation.
8. Read an Argo CD Application field by field and explain prune, selfHeal, targetRevision and ignoreDifferences in terms of the reconciliation loop.

2. Prerequisites

* kubectl v1.27 or later (it must embed Kustomize v5 for the labels: and patches: syntax used here). Check:

```
kubectl version --client
```

```
Client Version: v1.31.0
Kustomize Version: v5.4.2
```

If Kustomize Version reads v4.x, the labels: transformer in the overlays will be rejected. Upgrade kubectl, or substitute the deprecated commonLabels: field.

* A single-node kind cluster for Steps 7 and 9 only. Steps 3-6 and 8 need no cluster at all.

* Labs 17-20 completed.

* Terminal at this lab folder:

```
cd courseware/labs/lab-21-helm-kustomize-gitops
ls
```

```
README.md  brief.json  chart      data      gitops     kustomize  manifests
verification
```

3. Scenario

Kallang Freight Pte Ltd has spent Day 4 hardening TrackLane's storage (Labs 17-19) and documenting the control plane that runs it (Lab 20). The last gap is how changes actually reach the cluster.

TrackLane has outgrown its deployment process. There is one directory of YAML, and to ship to the dev cluster an engineer copies it, edits the replica count, edits the log level, edits the feature flag, and trims the shipping-lane dataset to the four domestic routes so the dev fixtures load faster.

Last Thursday someone copied prod → dev, changed the replica count and the log level, and missed the feature flag. FEATURE_LANE_ETA=true went to production, the lane-ETA endpoint hit a service that is not deployed there, and the customer portal returned 500s for eleven minutes.

The post-incident action is not "be more careful". It is: there must be exactly one place where dev and prod differ, and it must be reviewable in a pull request.

Your job this session is to build that in both of the two mainstream ways, decide which TrackLane should adopt, and then look at the delivery model that removes the human from the apply step entirely.

The platform team has already written down the intended difference, in data/release-inventory.csv. That file is the contract the graded verification checks your renders against.

```
column -s, -t data/release-inventory.csv
```

environment	name_suffix	replicas	cpu_request	memory_request	lane_rows	log_level	
feature_lane_eta							
dev	-dev	1	50m	64Mi	4	debug	true
prod	-prod	3	100m	128Mi	9	warn	false

4. Step-by-step procedure

Step 1 - Namespace

```
kubectl apply -f manifests/00-namespace.yaml
```

```
namespace/kcna-lab21 created
```

Step 2 - Deploy the "before" state and feel the problem

```
kubectl -n kcna-lab21 create configmap tracklane-handwritten-lanes \
  --from-file=shipping-lanes.csv=data/shipping-lanes.csv
kubectl apply -f manifests/01-tracklane-handwritten.yaml
```

```
configmap/tracklane-handwritten-lanes created
configmap/tracklane-handwritten-config created
service/tracklane-handwritten created
deployment.apps/tracklane-handwritten created
```

```
kubectl -n kcna-lab21 rollout status deployment/tracklane-handwritten --timeout=180s
```

```
deployment "tracklane-handwritten" successfully rolled out
```

Now count the work involved in producing a dev variant of this:

```
grep -c 'tracklane-handwritten' manifests/01-tracklane-handwritten.yaml
grep -c 'app.kubernetes.io/instance: tracklane-handwritten' manifests/01-tracklane-handwritten.yaml
```

11
6

Eleven occurrences of the instance name, six of them inside label blocks, spread across three objects - and you have to change every one consistently, in a copy, without breaking the ones you must *not* change (the selector labels, which are immutable after creation). Then you must remember the feature flag.

That is the defect. Both tools below attack it, from opposite directions:

	Helm	Kustomize
Mechanism	Templating. Source files are Go templates; values are substituted at render time	Patching. Source files are plain YAML; overlays declare deltas
Source is valid YAML?	No - {{ }} breaks every YAML tool	Yes - editors, kubeconform and kubectl apply all work on the base
Conditional resources	Yes - {{- if }} can create or omit a whole object	No - you can patch a resource, not conjure one
Distribution to third parties	Yes - versioned, packaged, registry-hosted charts	Poorly - Kustomize has no package format or registry
Release lifecycle	Yes - install/upgrade/rollback/history, state in a Secret	No - it renders; kubectl apply does the rest
Built into kubectl	No	Yes - kubectl kustomize, kubectl apply -k
Failure mode	A logic bug renders invalid YAML at deploy time	A patch silently matches nothing and does nothing

Step 3 - Read the Helm chart

```
find chart -type f | sort
```

```
chart/.helmignore
chart/Chart.yaml
chart/templates/NOTES.txt
chart/templates/_helpers.tpl
chart/templates/configmap.yaml
chart/templates/deployment.yaml
chart/templates/service.yaml
chart/values.yaml
```

That is a complete, valid chart. Every file earns its place:

Path	Purpose
Chart.yaml	Required. Chart metadata. Its presence is what makes a directory a chart
values.yaml	Required in practice. The default values, and the chart's public interface
templates/*.yaml	Rendered into Kubernetes objects
templates/_helpers.tpl	Leading underscore ☐ not rendered. Defines named templates used by the others

Path	Purpose
templates/NOTES.txt	Printed after helm install. Not an object; helm template does not emit it
.helmignore	Excludes files from helm package
charts/	*(absent here)* Vendored subcharts, populated by helm dependency update

version versus appVersion - the most commonly muddled pair in Helm:

```
grep -E '^(version|appVersion|apiVersion|type|kubeVersion):' chart/Chart.yaml
```

```
apiVersion: v2
type: application
version: 0.3.0
appVersion: "1.27"
```

* version: 0.3.0 - the version of the chart. Bump it when templates or defaults change. Helm enforces semver.

* appVersion: "1.27" - the version of the software being deployed. Pure metadata; Helm never uses it to select an image unless a template explicitly does. Quoted, or YAML reads 1.27 as a float and drops the trailing zero on 1.20.

* apiVersion: v2 - Helm 3. v1 is a Helm 2 chart.

Now the helpers - this is where the reusable logic lives:

```
grep -n '{{- define' chart/templates/_helpers.tpl
```

```
14:{{- define "tracklane.name" -}}
24:{{- define "tracklane.fullname" -}}
41:{{- define "tracklane.chart" -}}
52:{{- define "tracklane.selectorLabels" -}}
62:{{- define "tracklane.labels" -}}
76:{{- define "tracklane.validateImage" -}}
```

The selectorLabels / labels split is the single most important idea in this file.

```
sed -n '45,56p' chart/templates/_helpers.tpl
```

```
{{/*
SELECTOR labels. These go into Deployment.spec.selector.matchLabels, which is
IMMUTABLE after creation - so this set must stay minimal and must never
include anything that changes between releases (no version, no chart version).
Getting this wrong is the single most common cause of a chart that cannot be
upgraded in place.
*/}}
{{- define "tracklane.selectorLabels" -}}
app.kubernetes.io/name: {{ include "tracklane.name" . }}
app.kubernetes.io/instance: {{ .Release.Name }}
app.kubernetes.io/component: web
{{- end -}}
```

A chart that put app.kubernetes.io/version into the selector would break on the first version bump with field is immutable, and the only fix would be deleting and recreating the Deployment.

Step 4 - Render the chart for two environments

No helm? Read this step and skip the commands. The outputs shown are what the chart produces; checks.sh reports [SKIP] for these and still passes. Everything from Step 5 onward works with kubectl alone.

Render dev:

```
helm template tracklane-dev ./chart \
  --namespace kcna-lab21 \
  -f data/values-dev.yaml \
  --set-file lanes.csv=data/shipping-lanes.csv > /tmp/tracklane-dev.yaml
grep -E '^kind:' /tmp/tracklane-dev.yaml
```

```
kind: ConfigMap
kind: ConfigMap
kind: Service
kind: Deployment
```

Render prod:

```
helm template tracklane-prod ./chart \
  --namespace kcna-lab21 \
  -f data/values-prod.yaml \
  --set-file lanes.csv=data/shipping-lanes.csv > /tmp/tracklane-prod.yaml
grep -E '^kind:' /tmp/tracklane-prod.yaml
```

```
kind: PodDisruptionBudget
kind: ConfigMap
kind: ConfigMap
kind: Service
kind: Deployment
```

Four objects in dev, five in prod. The PodDisruptionBudget exists only because data/values-prod.yaml sets podDisruptionBudget.enabled: true and the template wraps it in {{- if }}. That is the capability Kustomize does not have - Kustomize can patch an object into a different shape, but it cannot make one appear or disappear.

On the ordering. Helm emits manifests in its own install order (a fixed kind precedence: namespaces and policy objects first, then config, then services, then workloads), not in source-file order. That is why PodDisruptionBudget appears first even though it is written at the bottom of templates/deployment.yaml. Do not read anything into the sequence.

Now the artefact a reviewer actually reads:

```
diff <(grep -E 'replicas:|LOG_LEVEL|FEATURE_LANE_ETA|CACHE_TTL|cpu:|memory:|^kind:' /tmp/
tracklane-dev.yaml) \
  <(grep -E 'replicas:|LOG_LEVEL|FEATURE_LANE_ETA|CACHE_TTL|cpu:|memory:|^kind:' /tmp/
tracklane-prod.yaml)
```

```

2,4c2,4
<  CACHE_TTL_SECONDS: "5"
<  FEATURE_LANE_ETA: "true"
<  LOG_LEVEL: "debug"
---
>  CACHE_TTL_SECONDS: "300"
>  FEATURE_LANE_ETA: "false"
>  LOG_LEVEL: "warn"
9c9
<  replicas: 1
---
>  replicas: 3
11,14c11,15
<    cpu: 200m
<    memory: 128Mi
<    cpu: 50m
<    memory: 64Mi
---
>    cpu: 500m
>    memory: 256Mi
>    cpu: 100m
>    memory: 128Mi
> kind: PodDisruptionBudget

```

Every line of that diff traces to one line in data/values-dev.yaml or data/values-prod.yaml. The environment difference is now a reviewable file, not a copy-paste ritual - which is exactly the post-incident action item.

Values precedence, lowest to highest - you must be able to state this:

```
chart/values.yaml < -f file (left to right) < --set < --set-string / --set-file
```

Prove it:

```
helm template tracklane-dev ./chart -f data/values-dev.yaml \
  --set replicaCount=7 | grep -m1 'replicas:'
```

```
replicas: 7
```

--set beat the values file, which beat the chart default of 2.

Confirm the dataset really travelled from data/:

```
grep -c '^    SG-' /tmp/tracklane-prod.yaml
echo "source rows: $(( $(wc -l < data/shipping-lanes.csv) - 1 ))"
```

```
9
source rows: 9
```

--set-file read the whole CSV off disk into .Values.lanes.csv, and the ConfigMap template embedded it with nindent 4.

The checksum/config annotation - find it:

```
grep -A1 'annotations:' /tmp/tracklane-prod.yaml | grep checksum
```

```
checksum/config: 3f7a1e5c9b02d846af1739e5c0d2b68475fa03c91d6e847b25c0af9e1d637208
```

```
grep -n 'checksum/config' chart/templates/deployment.yaml
```

```
33:         checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . |
sha256sum }}
```

The failure it prevents. Without it, helm upgrade with a changed logLevel updates the ConfigMap and leaves the Pod template byte-identical. The Deployment controller sees no change, does not roll, and your Pods serve the old config indefinitely - because envFrom values are read once at container start. With the annotation, the config hash is part of the Pod template, so any config change forces a rollout. Four lines; catches an entire class of "I deployed it but nothing changed" incidents.

Finally, lint the chart:

```
helm lint ./chart
```

```
==> Linting ./chart
[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed
```

Step 5 - Read the Kustomize base

Everything from here works with kubectl alone.

```
find kustomize -type f | sort
```

```
kustomize/base/deployment.yaml
kustomize/base/files/shipping-lanes.csv
kustomize/base/kustomization.yaml
kustomize/base/service.yaml
kustomize/overlays/dev/files/shipping-lanes.csv
kustomize/overlays/dev/kustomization.yaml
kustomize/overlays/dev/patch-resources.yaml
kustomize/overlays/prod/kustomization.yaml
kustomize/overlays/prod/patch-probe-timing.yaml
kustomize/overlays/prod/patch-resources.yaml
```

The defining property of the base - it is ordinary YAML:

```
kubectl apply --dry-run=client -f kustomize/base/deployment.yaml
```

```
deployment.apps/tracklane created (dry run)
```

```
/opt/homebrew/bin/kubeconform -strict -summary kustomize/base/deployment.yaml kustomize/base/
service.yaml
```

```
Summary: 2 resources found in 2 files - Valid: 2, Invalid: 0, Errors: 0, Skipped: 0
```

You cannot do either of those to a Helm template - `{{ .Values.replicaCount }}` is not valid YAML. That is the fundamental trade.

Now the base's kustomization.yaml:

```
cat kustomize/base/kustomization.yaml | grep -vE '^s*#|^$'
```

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
- deployment.yaml
- service.yaml
labels:
- pairs:
    app.kubernetes.io/managed-by: kustomize
    kcna.tertiaryinfotech.com/lab: "21"
  includeSelectors: false
commonAnnotations:
  tracklane.kallangfreight.sg/owner: platform-engineering
configMapGenerator:
- name: tracklane-config
  literals:
  - LOG_LEVEL=info
  - FEATURE_LANE_ETA=false
  - CACHE_TTL_SECONDS=60
  - ENVIRONMENT=base
- name: tracklane-lanes
  files:
  - shipping-lanes.csv=files/shipping-lanes.csv
generatorOptions:
  disableNameSuffixHash: false
```

Field	What it does
resources	The plain YAML files to include
labels ... includeSelectors: false	Adds labels to metadata.labels only. Setting true would also write them into spec.selector - an immutable field
commonAnnotations	Same idea for annotations
configMapGenerator	Builds ConfigMaps from literals or files, and appends a content hash to the name
generatorOptions.disableNameSuffixHash: false	Keeps the hash on. Turning it off reintroduces exactly the bug checksum/config fixes in Helm

Step 6 - Render both overlays

```
kubectrl kustomize kustomize/overlays/dev | grep -E '^kind:|^ name:|^ replicas:|^ LOG_LEVEL|^ FEATURE_LANE_ETA'
```

```
kind: ConfigMap
  name: tracklane-config-dev-m889h8f92h
  FEATURE_LANE_ETA: "true"
  LOG_LEVEL: debug
kind: ConfigMap
  name: tracklane-lanes-dev-6gmkh6tdbh
kind: Service
  name: tracklane-dev
kind: Deployment
  name: tracklane-dev
  replicas: 1
```

```
kubectl kustomize kustomize/overlays/prod | grep -E '^kind:|^ name:|^ replicas:|^ LOG_LEVEL|^ FEATURE_LANE_ETA'
```

```
kind: ConfigMap
  name: tracklane-config-prod-8c5db4d86b
  FEATURE_LANE_ETA: "false"
  LOG_LEVEL: warn
kind: ConfigMap
  name: tracklane-lanes-prod-t7f75mc2d4
kind: Service
  name: tracklane-prod
kind: Deployment
  name: tracklane-prod
  replicas: 3
```

Check the whole contract at once against the platform team's inventory:

```
for env in dev prod; do
  OUT=$(kubectl kustomize "kustomize/overlays/$env")
  printf '%-5s replicas=%s cpu=%s lanes=%s image=%s probe=%s\n' "$env" \
    "$(printf '%s' "$OUT" | grep -m1 '^ replicas:' | tr -d ' ' | cut -d: -f2)" \
    "$(printf '%s' "$OUT" | grep -A2 'requests:' | grep -m1 'cpu:' | tr -d ' ' | cut -d: -f2)" \
    "$(printf '%s' "$OUT" | grep -c '^ SG-')" \
    "$(printf '%s' "$OUT" | grep -m1 'image:' | tr -d ' ' | cut -d: -f2)" \
    "$(printf '%s' "$OUT" | grep -m1 'periodSeconds:' | tr -d ' ' | cut -d: -f2)"
done
```

```
dev replicas=1 cpu=50m lanes=4 image=nginx:1.27-alpine probe=5
prod replicas=3 cpu=100m lanes=9 image=docker.io/library/nginx:1.27-alpine probe=10
```

Compare against data/release-inventory.csv from Section 3 - every value matches. Five different mechanisms produced that line:

Difference	Mechanism	Where
tracklane-dev / tracklane-prod	nameSuffix	overlay kustomization.yaml
replicas 1 / 3	replicas: transformer (not a patch)	overlay kustomization.yaml
cpu 50m / 100m	strategic-merge patch	patch-resources.yaml
lanes 4 / 9	configMapGenerator with behavior: replace in dev; prod inherits the base	overlay kustomization.yaml
image bare / fully-qualified	images: transformer	prod kustomization.yaml
periodSeconds 5 / 10	JSON 6902 patch	patch-probe-timing.yaml

Strategic merge versus JSON 6902 - read them side by side:

```
cat kustomize/overlays/prod/patch-resources.yaml | grep -vE '^\\s*#|^$'
```

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: tracklane
spec:
  template:
    spec:
      containers:
        - name: web
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 256Mi

```

```
cat kustomize/overlays/prod/patch-probe-timing.yaml | grep -vE '^\\s*#|^$'
```

```

- op: replace
  path: /spec/template/spec/containers/0/readinessProbe/periodSeconds
  value: 10
- op: add
  path: /spec/template/spec/containers/0/readinessProbe/failureThreshold
  value: 3
- op: add
  path: /spec/template/spec/containers/0/readinessProbe/timeoutSeconds
  value: 3

```

	Strategic merge	JSON 6902
Shape	A YAML map with apiVersion/kind	A YAML list of {op, path, value}
Addresses list items	By merge key (containers merges on name)	By index (/containers/0)
Can remove a field	No	Yes (op: remove)
Breaks if a list is reordered	No	Yes, silently
Readability	High - looks like the object	Low - looks like a patch
Use it when	Almost always	You need remove, an exact index, or a field with no merge key

Kustomize infers which one you meant from the file's shape. There is no flag.

Finally, prove the two overlays cannot interfere with each other in the shared namespace:

```

for env in dev prod; do
  printf '%s selector: ' "$env"
  kubectl kustomize "kustomize/overlays/$env" \
    | grep -A4 '^ selector:' | grep 'instance:' | head -1 | tr -d ' '
done

```

```

dev selector: app.kubernetes.io/instance:tracklane-dev
prod selector: app.kubernetes.io/instance:tracklane-prod

```

That is includeSelectors: true in the overlays doing its job. With false, both Deployments would carry the base's selector, both Services would match all six Pods, and the two "environments" would silently merge.

Step 7 - Apply the dev overlay to the cluster

kubectl apply -k runs the same render and pipes it straight to apply:

```
kubectl apply -k kustomize/overlays/dev
```

```
configmap/tracklane-config-dev-m889h8f92h created
configmap/tracklane-lanes-dev-6gmkh6tdbh created
service/tracklane-dev created
deployment.apps/tracklane-dev created
```

```
kubectl -n kcna-lab21 rollout status deployment/tracklane-dev --timeout=180s
```

```
deployment "tracklane-dev" successfully rolled out
```

```
kubectl -n kcna-lab21 get deployment,configmap -l app.kubernetes.io/instance=tracklane-dev
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/tracklane-dev	1/1	1	1	24s

NAME	DATA	AGE
configmap/tracklane-config-dev-m889h8f92h	4	24s
configmap/tracklane-lanes-dev-6gmkh6tdbh	1	24s

Confirm the config actually reached the container, and that the trimmed dev dataset is what is mounted:

```
kubectl -n kcna-lab21 exec deployment/tracklane-dev -- \
  sh -c 'echo "LOG_LEVEL=$LOG_LEVEL FEATURE_LANE_ETA=$FEATURE_LANE_ETA"; \
    echo "lanes=$(( $(wc -l < /usr/share/nginx/html/lanes/shipping-lanes.csv) - 1 ))"'
```

```
LOG_LEVEL=debug FEATURE_LANE_ETA=true
lanes=4
```

Four lanes, debug logging, feature flag on - the dev contract, delivered without anyone editing a copy of the base.

Now watch the generator hash do its job. Change a literal in the dev overlay:

```
sed -i.bak 's/CACHE_TTL_SECONDS=5$/CACHE_TTL_SECONDS=15/' kustomize/overlays/dev/kustomization.
yaml
kubectl apply -k kustomize/overlays/dev
```

```
configmap/tracklane-config-dev-f2c68b9047 created
configmap/tracklane-lanes-dev-6gmkh6tdbh unchanged
service/tracklane-dev unchanged
deployment.apps/tracklane-dev configured
```

A new ConfigMap name, and deployment configured. The hash changed, so the Deployment's Pod template changed, so it rolled:

```
kubectl -n kcna-lab21 rollout status deployment/tracklane-dev --timeout=180s
kubectl -n kcna-lab21 exec deployment/tracklane-dev -- printenv CACHE_TTL_SECONDS
```

```
deployment "tracklane-dev" successfully rolled out
15
```

Restore the file:

```
mv kustomize/overlays/dev/kustomization.yaml.bak kustomize/overlays/dev/kustomization.yaml
kubectl apply -k kustomize/overlays/dev >/dev/null
grep -n 'CACHE_TTL_SECONDS' kustomize/overlays/dev/kustomization.yaml
```

```
43:      - CACHE_TTL_SECONDS=5
```

Note the leftover. The old ConfigMap tracklane-config-dev-m889h8f92h is still in the namespace - apply creates the new one and nothing deletes the old. Kustomize alone has no garbage collection. kubectl apply --prune and, properly, a GitOps controller's prune: true are what solve this. That is the bridge to Step 8.

```
kubectl -n kcna-lab21 get configmap | grep tracklane-config-dev
```

```
tracklane-config-dev-f2c68b9047    4      2m
tracklane-config-dev-m889h8f92h    4      6m
```

Step 8 - GitOps: read the reconciliation contract

Nothing so far removed the human. Somebody still runs kubectl apply. That means:

- * The cluster's state depends on who ran what, from which laptop, at what time.
- * Nothing detects or corrects a manual kubectl edit at 02:00.
- * kubectl apply -k from a workstation needs cluster-admin on a human's credential.

GitOps inverts this. The four principles, as OpenGitOps states them:

1. Declarative - the system is described entirely by declarative state.
2. Versioned and immutable - that state lives in Git, with history and signatures.
3. Pulled automatically - agents **inside** the cluster fetch the state. Nothing is pushed in.
4. Continuously reconciled - agents constantly compare actual to desired and converge.

Point 3 is the security argument: the CI system never needs cluster credentials at all. Point 4 is the reliability argument.

Look at the manifest:

```
head -25 gitops/argocd-application.yaml
```

```
# =====
# Lab 21 / Step 8 - an Argo CD Application, for READING.
#
# ■■■ DO NOT APPLY THIS FILE. It is deliberately kept OUT of manifests/ so
#     that `kubectl apply -f manifests/` cannot pick it up by accident.
# ...
```

Confirm for yourself that it genuinely cannot be applied here:

```
kubectl apply --dry-run=client -f gitops/argocd-application.yaml
```

```
error: resource mapping not found for name: "tracklane-dev" namespace: "argocd" from "gitops/argocd-application.yaml": no matches for kind "Application" in version "argoproj.io/v1alpha1"
ensure CRDs are installed first
```

That error is the lesson. Argo CD is not special infrastructure - it is a controller that installs a CRD and then watches instances of it. Without the CRD, the API server has never heard of kind: Application. Every GitOps tool works this way.

Now read the four blocks that matter:

```
grep -vE '\s*#|\s*$|^# ' gitops/argocd-application.yaml | sed -n '1,30p'
```

```
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: tracklane-dev
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
  labels:
    tracklane.kallangfreight.sg/environment: dev
spec:
  project: default
  source:
    repoURL: https://github.com/tertiarycourses/TGS-2023039343-Kubernetes-and-Cloud-Native-Associate-KCNA-Training
    targetRevision: main
    path: courseware/labs/lab-21-helm-kustomize-gitops/kustomize/overlays/dev
    kustomize:
      namePrefix: ""
      commonAnnotations:
        tracklane.kallangfreight.sg/reconciled-by: argocd
  destination:
    server: https://kubernetes.default.svc
    namespace: kcna-lab21
```

Field	Meaning	The consequence of getting it wrong
metadata.namespace: argocd	The Application lives in Argo CD's namespace, not the target's	Argo CD only watches its own namespace by default; put it elsewhere and nothing happens, with no error
finalizers: [resources-finalizer...]	Deleting the Application also deletes what it created	Without it, deleting the Application orphans the workloads, which keep running unmanaged
spec.project	The AppProject restricting allowed repos, destinations and kinds	default permits everything - fine for a demo, wrong on a shared cluster
source.repoURL	Where the desired state lives. This is the whole of GitOps	-
source.targetRevision: main	A branch tracks the tip; a tag or SHA pins	A mutable branch makes "what is deployed?" unanswerable during an incident. Production should pin
source.path	The directory in the repo. Argo CD detects kustomization.yaml → runs kustomize; Chart.yaml → runs helm template	A wrong path renders nothing; see allowEmpty below

Field	Meaning	The consequence of getting it wrong
destination.server	https://kubernetes.default.svc = "the cluster Argo CD runs in"	-

And the reconciliation contract itself:

```
grep -A16 'syncPolicy:' gitops/argocd-application.yaml | grep -vE '^\\s*#'
```

```
syncPolicy:
  automated:
    prune: true
    selfHeal: true
    allowEmpty: false
  syncOptions:
    - CreateNamespace=true
    - ServerSideApply=true
    - RespectIgnoreDifferences=true
  retry:
    limit: 5
```

Setting	What the controller does	Why it matters
prune: true	Deletes cluster objects no longer present in Git	This is what fixes the orphaned ConfigMap you created in Step 7. Without prune, deleting a manifest from Git leaves the object running forever and Git stops being the source of truth
selfHeal: true	Reverts manual changes back to Git's state	The anti-drift mechanism. Someone scales a Deployment by hand; the controller scales it back and records the divergence
allowEmpty: false	Refuses to sync a render that produces zero resources	A typo'd path would otherwise be read as "Git says delete everything"
ServerSideApply=true	Applies with field-manager ownership tracking	Two controllers cannot silently fight over one field
ignoreDifferences: /spec/replicas	Concedes that field to another owner	Without it, an HPA changing replicas reads as permanent drift and selfHeal fights the HPA in a loop

The loop, in one sentence: *observe Git → render → compare to the live cluster → apply the difference → repeat forever.* There is no "deploy" step, and there is no push.

Flux expresses the identical model with different nouns:

```
grep -E '^(kind| name| interval| path| prune| targetNamespace):' gitops/flux-kustomization.yaml
```

```
kind: GitRepository
  name: tracklane
  interval: 1m
kind: Kustomization
  name: tracklane-dev
  interval: 5m
  path: ./courseware/labs/lab-21-helm-kustomize-gitops/kustomize/overlays/dev
  prune: true
  targetNamespace: kcna-lab21
```

Argo CD	Flux
Application.spec.source	a separate GitRepository object
Application.spec.destination	Kustomization.spec.targetNamespace
syncPolicy.automated.prune	Kustomization.spec.prune
syncPolicy.automated.selfHeal	drift detection + force
built-in polling	explicit spec.interval

Argo CD packs source, destination and policy into one object and ships a web UI. Flux decomposes them into composable controllers and leans on the CLI and Git. Both implement the same loop. Neither is "more GitOps".

Step 9 - Decide

For TrackLane, given the incident in Section 3:

Situation	Choose	Why
Internal app, 2-3 environments, small team	Kustomize	The base stays plain YAML; the overlay diff *is* the review artefact; nothing extra to install
Shipping software other organisations install	Helm	Versioned, packaged, registry-distributable; values.yaml is a documented public interface
Environments that need different sets of objects	Helm	Only templating can conditionally create a resource
Installing third-party software (Prometheus, cert-manager)	Helm	That is how upstream publishes it
Consuming a third-party chart but needing 3 fields the chart does not expose	Both - render the chart, then patch it with Kustomize	This is why Argo CD and Flux both support "Helm render, then Kustomize post-render"
Any of the above, more than one cluster or more than one operator	plus GitOps	Neither tool solves drift, audit or credential distribution. That is a delivery-model problem, not a packaging one

TrackLane's answer: Kustomize for the app, Helm for third-party dependencies, GitOps for delivery. The overlay diff is what a reviewer reads, and selfHeal is what stops the next 02:00 hand-edit.

5. Verification

This lab is graded on the static render. Run it with no cluster if you like:

```
bash verification/checks.sh
```

```

== Lab 21 verification - static render (no cluster required) ==
[PASS] chart/ contains Chart.yaml, values.yaml, _helpers.tpl and 3 templates
[PASS] Chart.yaml declares apiVersion v2, name tracklane and a semver version
[PASS] no ':latest' image tag anywhere in chart/, kustomize/ or manifests/
[PASS] kubectl kustomize renders overlays/dev
[PASS] kubectl kustomize renders overlays/prod
[PASS] both rendered overlays pass kubectl apply --dry-run=client
[PASS] overlays/dev matches data/release-inventory.csv (suffix -dev, 1 replicas, 50m/64Mi, 4
lanes, LOG_LEVEL=debug)
[PASS] overlays/prod matches data/release-inventory.csv (suffix -prod, 3 replicas, 100m/128Mi,
9 lanes, LOG_LEVEL=warn)
[PASS] the instance label was injected into selectors as well as metadata (includeSelectors:
true)
[PASS] the prod images transformer rewrote the image to a fully-qualified registry path
[PASS] dev inherited the base image reference unchanged
[PASS] the prod JSON 6902 patch changed readinessProbe.periodSeconds 5 -> 10
[PASS] generated ConfigMap carries a content hash (tracklane-config-dev-m889h8f92h) and the
Deployment reference was rewritten
[PASS] overlays/dev and overlays/prod render materially different output
[PASS] helm lint passes on chart/
[PASS] helm template renders the chart with data/values-dev.yaml
[PASS] helm template renders the chart with data/values-prod.yaml
[PASS] the values files drove replicaCount 1 (dev) and 3 (prod)
[PASS] the conditional PodDisruptionBudget renders in prod only
[PASS] the Deployment carries a checksum/config annotation
[PASS] --set-file carried all 9 rows of data/shipping-lanes.csv into the render
[PASS] the Argo CD Application lives in gitops/, not manifests/ (it needs a CRD)
[PASS] the Application declares repoURL, targetRevision, prune and selfHeal
[PASS] deployment/tracklane-dev is available in kcna-lab21 (applied from the dev overlay)
[PASS] no container in kcna-lab21 requests privileged: true
-----
25 passed, 0 failed, 0 skipped

```

Without helm installed the five Helm lines become [SKIP]; without a cluster the two cluster lines and the dry-run line become [SKIP]. Skips are not failures - the script exits 0:

```

-----
15 passed, 0 failed, 8 skipped

```

Full transcript, including both variants: verification/expected-output.md.

6. Failure injection

Failure A - a Kustomize patch that silently matches nothing

This is Kustomize's characteristic failure and it is far more dangerous than a Helm error, because nothing goes wrong. Helm fails loudly at render time; Kustomize just... does not apply your patch.

Break the patch target names in the prod overlay - the environment where the patches genuinely matter:

```

cp kustomize/overlays/prod/kustomization.yaml /tmp/prod-kustomization.bak
sed -i.tmp 's|^      name: tracklane$|      name: tracklane-web|' kustomize/overlays/prod/
kustomization.yaml
grep -A6 '^patches:' kustomize/overlays/prod/kustomization.yaml | grep -vE '^s*#'

```

```
patches:
- path: patch-resources.yaml
  target:
    group: apps
    version: v1
    kind: Deployment
    name: tracklane-web
```

Both patch targets in that file now name tracklane-web, which does not exist. Render it:

```
kubectl kustomize kustomize/overlays/prod >/dev/null; echo "render exit=$?"
```

```
render exit=0
```

Exit 0. No error. No warning. The render succeeded and both patches did nothing. Now look at what it produced:

```
kubectl kustomize kustomize/overlays/prod | grep -A3 'requests:'
kubectl kustomize kustomize/overlays/prod | grep 'periodSeconds:'
```

```
requests:
  cpu: 50m
  memory: 64Mi
securityContext:
  periodSeconds: 5
```

Production silently rendered with the base's resource floor and the base's probe timing. cpu: 100m never appeared; periodSeconds: 10 never appeared. There was no error, kubectl apply -k would have succeeded, and the only symptom would be a production web tier scheduled with half the CPU it was sized for - surfacing days later as latency.

Diagnosis, in order:

1. Suspect it whenever a value you *know* you set does not appear in the render. Kustomize does not tell you.
2. Diff the render against the base. This is the reliable detector:

```
diff <(kubectl kustomize kustomize/base 2>/dev/null | grep -A3 'requests:') \
  <(kubectl kustomize kustomize/overlays/prod | grep -A3 'requests:') \
  && echo "NO DIFFERENCE - the prod overlay is not changing resources at all"
```

```
NO DIFFERENCE - the prod overlay is not changing resources at all
```

3. Compare the patch target against the base resource's name - remembering that targets match the name before nameSuffix:

```
grep -m1 -A1 '^metadata:' kustomize/base/deployment.yaml
grep -A6 '^patches:' kustomize/overlays/prod/kustomization.yaml | grep 'name:'
```

```
metadata:
  name: tracklane
  name: tracklane-web
```

tracklane ≠ tracklane-web. There is the bug.

The guard. Be clear about this: Kustomize has no built-in option that turns an unmatched patch target into an error. A patches entry whose target selects nothing is not a syntax error and not a warning; it is simply a no-op. Do not go

looking for a flag.

The guard is therefore procedural, and it is the same one you would use for any renderer: assert on the render in CI. That is exactly what verification/checks.sh does when it compares both overlays against data/release-inventory.csv. Run it now with the sabotage still in place:

```
bash verification/checks.sh 2>&1 | grep -E '^\[FAIL\]|passed,'
```

```
[FAIL] overlays/prod: expected cpu request 100m
[FAIL] expected readinessProbe.periodSeconds 5 in dev and 10 in prod
13 passed, 2 failed, 8 skipped
```

Both silent no-ops were caught, and the script exits non-zero. That is the whole argument for grading on the **render** rather than on "did the apply succeed". Note the counts assume no helm and no cluster; with both present you would see the same two [FAIL] lines among more passes.

Restore the file and confirm you are back to green:

```
cp /tmp/prod-kustomization.bak kustomize/overlays/prod/kustomization.yaml
rm -f kustomize/overlays/prod/kustomization.yaml.tmp /tmp/prod-kustomization.bak
bash verification/checks.sh 2>&1 | tail -2
```

```
-----
15 passed, 0 failed, 8 skipped
```

Failure B - the chart's own guard rejects an unpinned tag

Requires helm. Read it if you do not have helm - the mechanism is fail in _helpers.tpl.

```
helm template tracklane-dev ./chart -f data/values-dev.yaml --set image.tag=latest
```

```
Error: execution error at (tracklane/templates/deployment.yaml:50:20): image.tag must not be
'latest' - pin an immutable tag or a digest
```

Use --debug flag to render out invalid YAML

Diagnosis. The error names the file, the line and the reason. _helpers.tpl uses Helm's fail function:

```
sed -n '76,82p' chart/templates/_helpers.tpl
```

```
{{- define "tracklane.validateImage" -}}
{{- $tag := required "image.tag is required and must be pinned" .Values.image.tag -}}
{{- if eq $tag "latest" -}}
{{- fail "image.tag must not be 'latest' - pin an immutable tag or a digest" -}}
{{- end -}}
{{- printf "%s:%s" .Values.image.repository $tag -}}
{{- end -}}
```

Prove required fires too:

```
helm template tracklane-dev ./chart -f data/values-dev.yaml --set image.tag=null
```

```
Error: execution error at (tracklane/templates/deployment.yaml:50:20): image.tag is required
and must be pinned
```

Contrast the two failure modes, and remember it:

	Helm	Kustomize
Bad input	Fails at render time with a file, line and message	Renders successfully and quietly omits your change
Guard available	required, fail, JSON-schema values.schema.json	None built in - you must assert on the render
Where it bites	In CI, immediately	In production, later

Neither is safe without a CI step that checks the `*render*`. Helm just makes more mistakes impossible to ignore.

7. Troubleshooting

Symptom	Likely cause	Diagnostic command	Fix	
A patch has no effect; the render succeeds with exit 0	The patch target name/kind/group does not match any resource. Targets match the base name, before nameSuffix	diff <(kubectl kustomize base) <(kubectl kustomize overlays/<env>)	Correct the target to the base's metadata.name; assert on the render in CI	
kubectl kustomize fails with json: unknown field "labels"	kubectl embeds Kustomize v4, which predates the labels: transformer	kubectl version --client	Upgrade kubectl to v1.27+, or fall back to the deprecated commonLabels:	
security; file './../data/x.csv' is not in or below '...'	A generator references a file outside the kustomization root; Kustomize forbids this by default	grep -rn 'files:' kustomize/	Keep generator inputs inside the overlay directory (as this lab does), or accept --load-restrictor LoadRestrictionsNone and its risks	
Config changed but the Pods still serve the old values	envFrom reads a ConfigMap once at container start, and nothing changed the Pod template	Helm: grep checksum/config. Kustomize: check disableNameSuffixHash	Keep the generator hash on (Kustomize) or the checksum/config annotation (Helm)	
Old hashed ConfigMaps accumulate in the namespace	kubectl apply -k creates the new object and never deletes the old	`kubectl -n kcna-lab21 get cm \	grep tracklane-config`	Delete the stale ones by name, or adopt a GitOps controller with prune: true
helm upgrade fails with field is immutable on spec.selector	A changing label (version, chart version) leaked into the selector template	`helm template ... \	grep -A4 'selector:'`	Split selector labels from descriptive labels, as _helpers.tpl does. The only in-place remedy is to delete and recreate
helm template fails with nil pointer evaluating interface {}	A template dereferences a key that has no default in values.yaml	helm template ./chart --debug	Add the default to values.yaml, or guard with {{- with }} / default	
Both "environments" behave as one; Services return each other's Pods	The environment label was added with includeSelectors: false, so the selectors are identical	`kubectl kustomize overlays/<env> \	grep -A4 '^ selector:'`	Set includeSelectors: true for the instance label, as the overlays here do
no matches for kind "Application" in version "argoproj.io/v1alpha1"	Argo CD is not installed, so its CRD does not exist	`kubectl get crd \	grep argoproj`	Expected in this lab. On a real cluster, install Argo CD first
Argo CD reports permanent OutOfSync on replicas	An HPA owns that field and the controller keeps reverting it	kubectl get hpa -A	Add ignoreDifferences for /spec/replicas, as the sample Application does	

8. Cleanup

Everything this lab created is inside kcna-lab21. No cluster-scoped objects, no CRDs, no controllers, no Helm releases - helm template renders to stdout and never contacts a cluster.

```
kubectl delete namespace kcna-lab21 --wait=true
```

```
namespace "kcna-lab21" deleted
```

Confirm no Helm release was ever created (if you have helm):

```
helm list --all-namespaces | grep tracklane || echo "no tracklane helm release exists"
```

```
no tracklane helm release exists
```

Remove the local render artefacts and any backup files from the failure injection:

```
rm -f /tmp/tracklane-dev.yaml /tmp/tracklane-prod.yaml /tmp/prod-kustomization.bak
rm -f kustomize/overlays/*/kustomization.yaml.tmp kustomize/overlays/*/kustomization.yaml.bak
echo "local artefacts removed"
```

```
local artefacts removed
```

Confirm the lab tree is back to its committed state and still renders:

```
kubectl kustomize kustomize/overlays/dev >/dev/null && \
kubectl kustomize kustomize/overlays/prod >/dev/null && \
echo "both overlays still render"
kubectl get namespace kcna-lab21
```

```
both overlays still render
Error from server (NotFound): namespaces "kcna-lab21" not found
```

9. What you learned

- * Copy-paste-and-edit does not scale past one environment. The failure it produces is not "messy YAML" - it is a config value silently taking the wrong value in production, which is exactly the incident in Section 3.
- * A Helm chart is Chart.yaml + values.yaml + templates/. version is the chart's; appVersion is the software's. Files beginning with _ define named templates and render to nothing. NOTES.txt prints after install and is not an object.
- * Values precedence is values.yaml < -f < --set < --set-file, and you proved it by overriding a replica count from the command line.
- * Separate selector labels from descriptive labels. spec.selector is immutable; a chart that puts a version into it cannot be upgraded in place. _helpers.tpl shows the correct split.
- * checksum/config forces a rollout when configuration changes. Kustomize achieves the same thing with generator name hashes - and turning disableNameSuffixHash: true on reintroduces the bug.
- * Helm templates; Kustomize patches. Kustomize bases stay valid YAML, so kubeconform, editors and kubectl apply all work on them. Only Helm can conditionally *create* a resource - which is why prod renders a PodDisruptionBudget and dev does not.

- * Strategic-merge patches address list items by merge key; JSON 6902 patches address them by index and can remove. Prefer strategic merge; reach for 6902 only when you must.
- * Kustomize's characteristic failure is silence. A patch whose target matches nothing renders exit 0 and changes nothing. The defence is to assert on the `*render*` in CI - which is why this lab is graded on `kubectl kustomize` output compared against `data/release-inventory.csv`, not on whether an apply succeeded.
- * GitOps is a delivery model, not a packaging tool. Desired state in Git, pulled by an in-cluster agent, continuously reconciled. `prune` makes Git authoritative about deletion; `selfHeal` corrects manual drift; `ignoreDifferences` concedes fields to other controllers such as an HPA.
- * A GitOps controller is an ordinary controller with a CRD. You confirmed this the honest way - by watching the API server reject kind: Application because the CRD is not installed.
- * Argo CD and Flux implement the same loop with different object models. Neither is more "GitOps" than the other.

10. Further reading

- * Helm documentation - `*Charts*` (the authoritative reference for chart structure):
<<https://helm.sh/docs/topics/charts/>>
- * Helm documentation - `*Values files and precedence*`: <https://helm.sh/docs/chart_template_guide/values_files/>
- * Helm documentation - `*Named templates*` (`_helpers.tpl`):
<https://helm.sh/docs/chart_template_guide/named_templates/>
- * Helm documentation - `*Chart best practices: labels and annotations*`:
<https://helm.sh/docs/chart_best_practices/labels/>
- * Helm - installation: <<https://helm.sh/docs/intro/install/>>
- * Kustomize documentation - `*Kustomization file reference*` (labels, patches, replicas, images, generators):
<<https://kubectl.docs.kubernetes.io/references/kustomize/kustomization/>>
- * Kubernetes documentation - `*Declarative management of Kubernetes objects using Kustomize*`:
<<https://kubernetes.io/docs/tasks/manage-kubernetes-objects/kustomization/>>
- * RFC 6902 - `*JSON Patch*` (the `op/path/value` format): <<https://datatracker.ietf.org/doc/html/rfc6902>>
- * Kubernetes documentation - `*Recommended labels*`:
<<https://kubernetes.io/docs/concepts/overview/working-with-objects/common-labels/>>
- * OpenGitOps - `*GitOps Principles v1.0*` (a CNCF project): <<https://opengitops.dev/>>
- * Argo CD documentation - `*Application specification reference*`:
<<https://argo-cd.readthedocs.io/en/stable/user-guide/application-specification/>>
- * Flux documentation - `*Kustomization*` and `*GitRepository*`:
<<https://fluxcd.io/flux/components/kustomize/kustomizations/>>
- * CNCF KCNA Curriculum - `*Cloud Native Application Delivery*`: <<https://github.com/cncf/curriculum>>